LECTURE · DEEP LEARNING

Neural Network Foundations

From a single neuron to a trained network

Intuition first, visuals throughout, math only where it earns its place

Neurons and activations · Layers and expressive power · Loss and gradient descent
Backpropagation · Overfitting and practical defaults

Three ideas explain almost all of deep learning

Everything later in the course is a variation on these three

1

A neuron is a weighted vote followed by a bend

Linear scoring plus a nonlinear activation is the only building block.

2

Stacking bends lets a network draw any shape

Each layer reshapes the space so the next layer's job becomes simpler.

3

Learning means sliding downhill on a loss surface

Backpropagation computes the slope; gradient descent takes the step.

Model = architecture (shape of functions) · Training = search (which function)

Name the iris species from four measurements

Iris dataset (Fisher 1936): setosa, versicolor, virginica





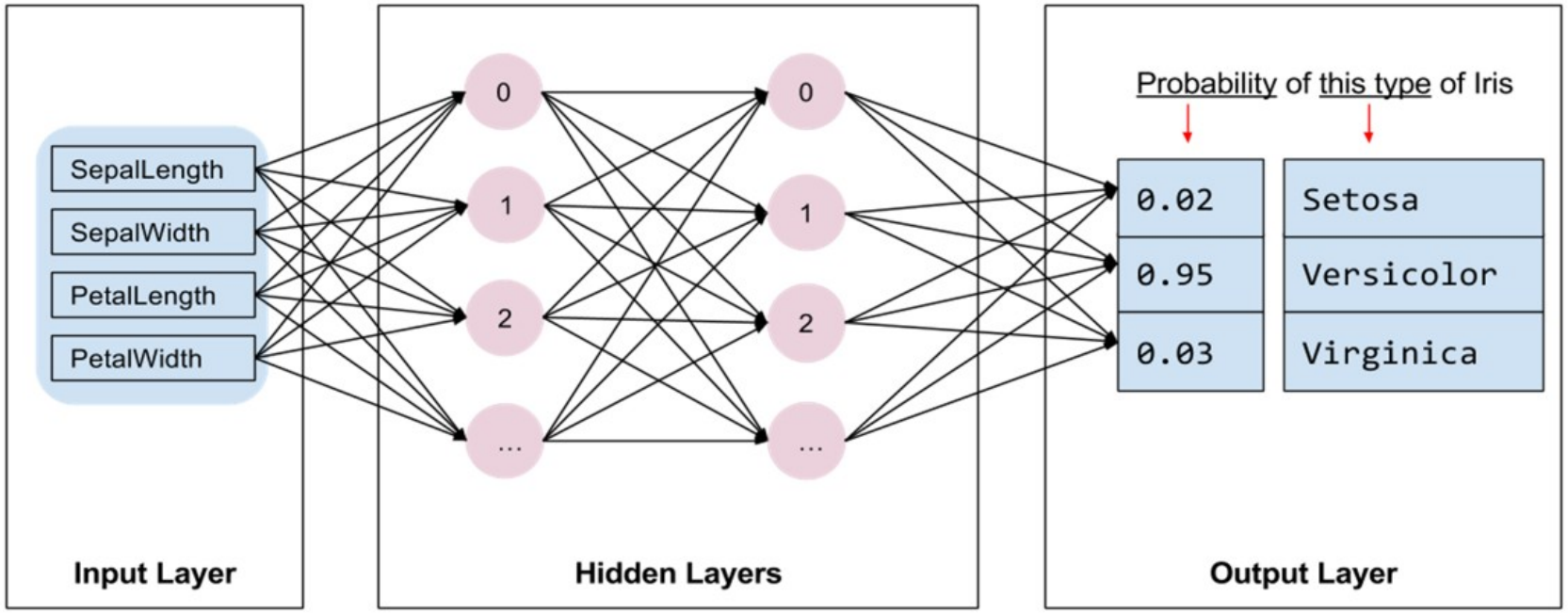


The task

- Inputs: sepal length, sepal width, petal length, petal width (cm)
- Output: one of three species; 150 flowers, 50 per species

Four measurements in, three probabilities out

Two hidden layers of 10 ReLU units; softmax makes the 3 outputs sum to 1



Keras code · fully connected = Dense layers

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(10, activation="relu", input_shape=(4,)),   # 4 inputs
    tf.keras.layers.Dense(10, activation="relu"),
    tf.keras.layers.Dense(3)])                                        # 3 class scores -> softmax
```

01

PART 1 · ANATOMY

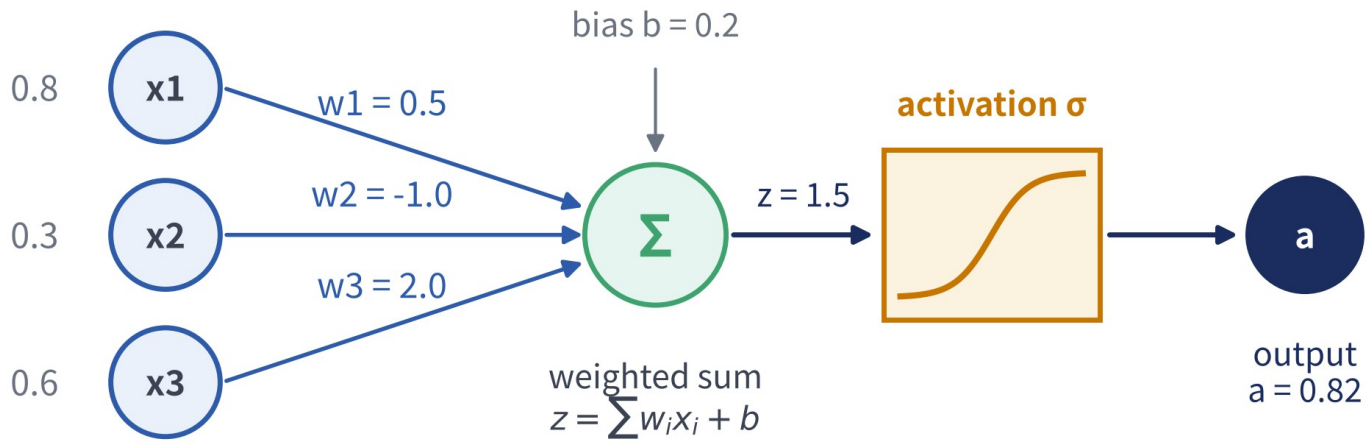
The Building Blocks of a Network

Neurons, activations, layers and why depth matters

A neuron weighs evidence, then decides how to fire

Worked example: inputs (0.8, 0.3, 0.6), weights (0.5, −1.0, 2.0), bias 0.2

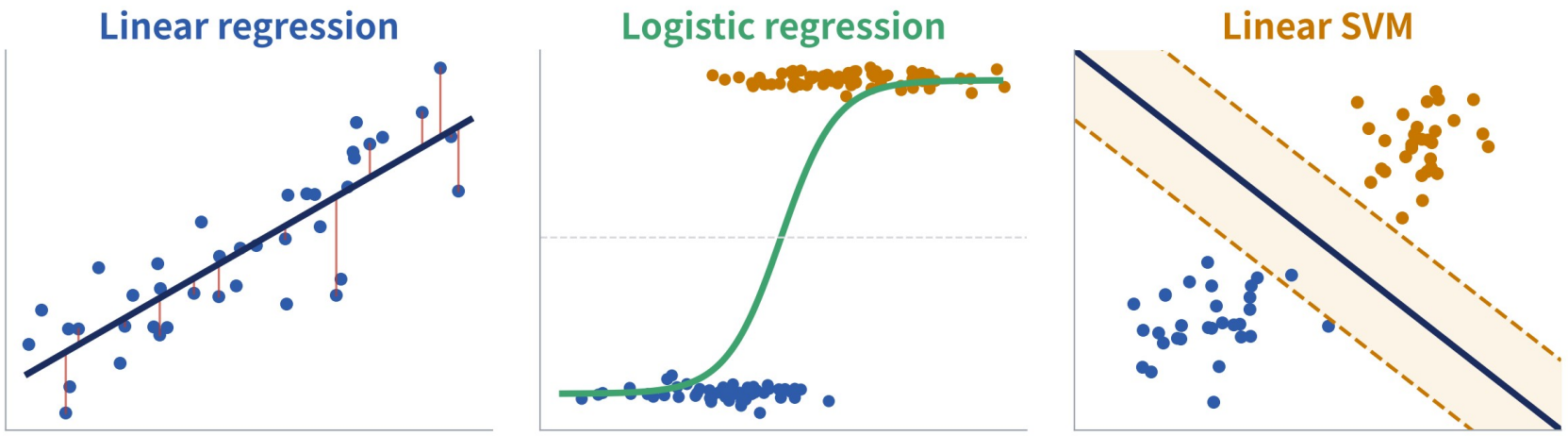
$$a = \sigma\Big(\underbrace{w_1x_1 + w_2x_2 + \cdots + w_nx_n}_{\text{weighted evidence}} + \underbrace{b}_{\text{threshold}}\Big) = \sigma(\mathbf{w}^\top\mathbf{x} + b)$$



One neuron, three objectives, three classic models

Same linear score z = $w^T x + b$; only the objective changes (y = ±1 for classification)

$$z = \mathbf{w}^\top \mathbf{x} + b \quad \Longrightarrow \quad \begin{cases} \text{Linear regression:} & \min \ \frac{1}{N}\sum (y_i - z_i)^2 \\ \text{Logistic regression:} & \min \ \frac{1}{N}\sum \log\left(1 + e^{-y_i z_i}\right) \\ \text{Linear SVM:} & \min \ \frac{1}{N}\sum \max(0,\ 1 - y_i z_i) + \lambda \|\mathbf{w}\|^2 \end{cases}$$



fit a line, squared residuals

output a probability, log loss

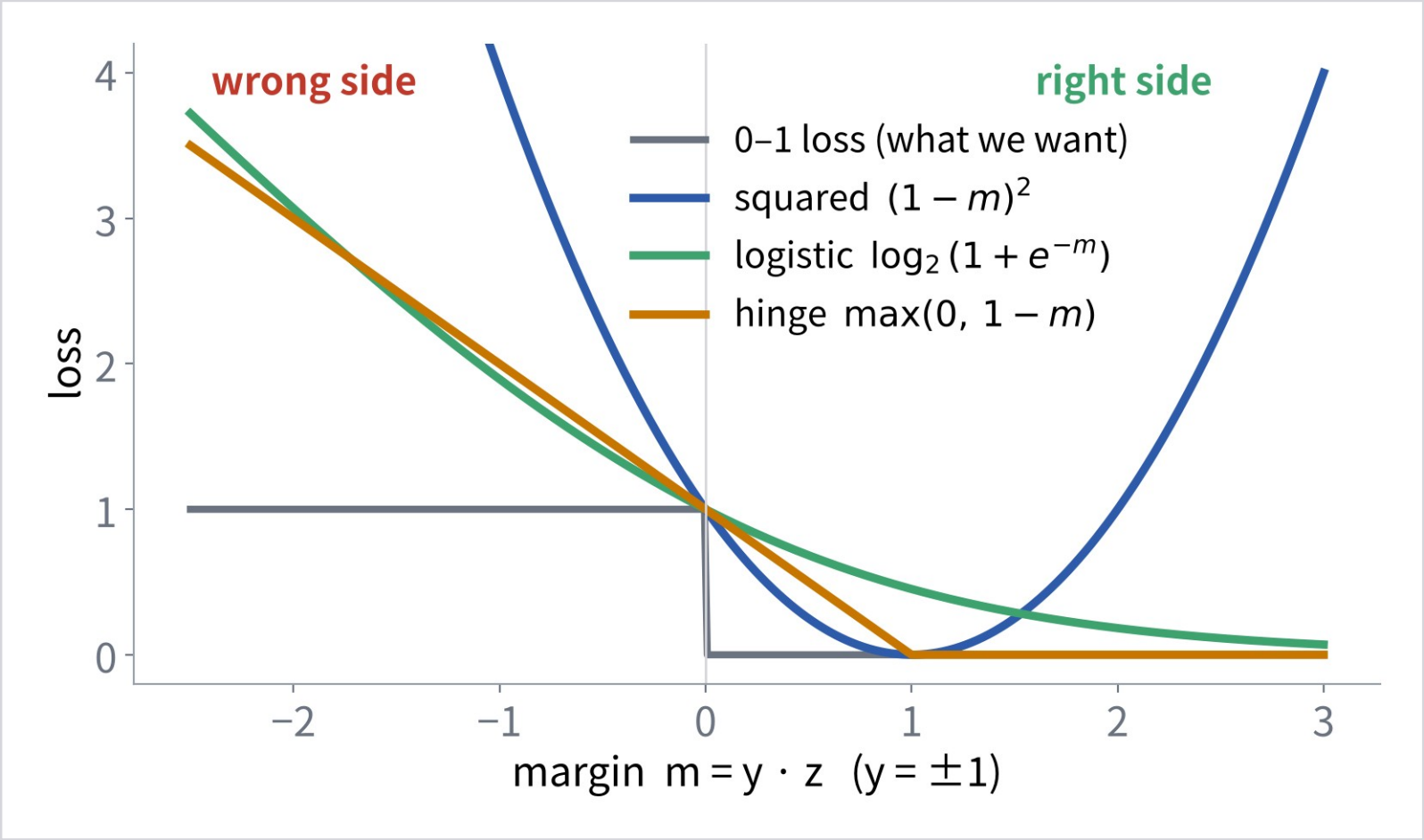
widest margin, hinge loss

Each loss punishes margin mistakes differently

Margin m = y·z: positive means correct side, larger means more confident

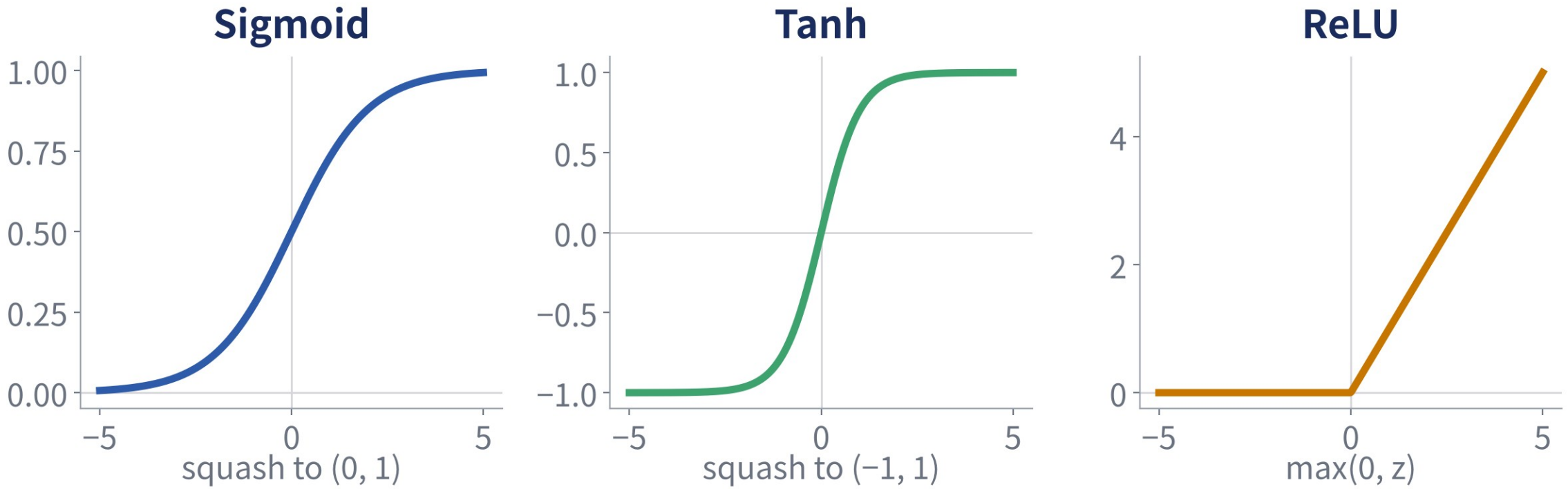
How to read

- All approximate 0 –1 loss
- Squared: hurts even when right
- Logistic: never fully zero
- Hinge: zero past margin 1
- Neural net = learn z too



Activation functions bend the straight line

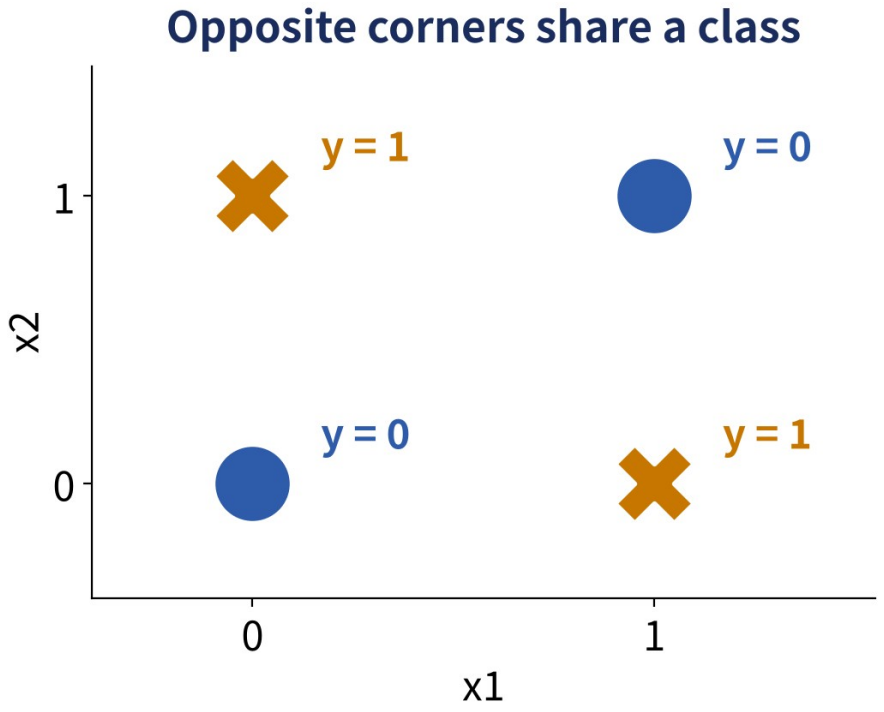
Without them, any stack of layers collapses into a single linear model



XOR outputs 1 only when the two inputs differ

Exclusive OR is a basic logic gate: true when exactly one input is true

x1	x2	OR	AND	XOR
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0



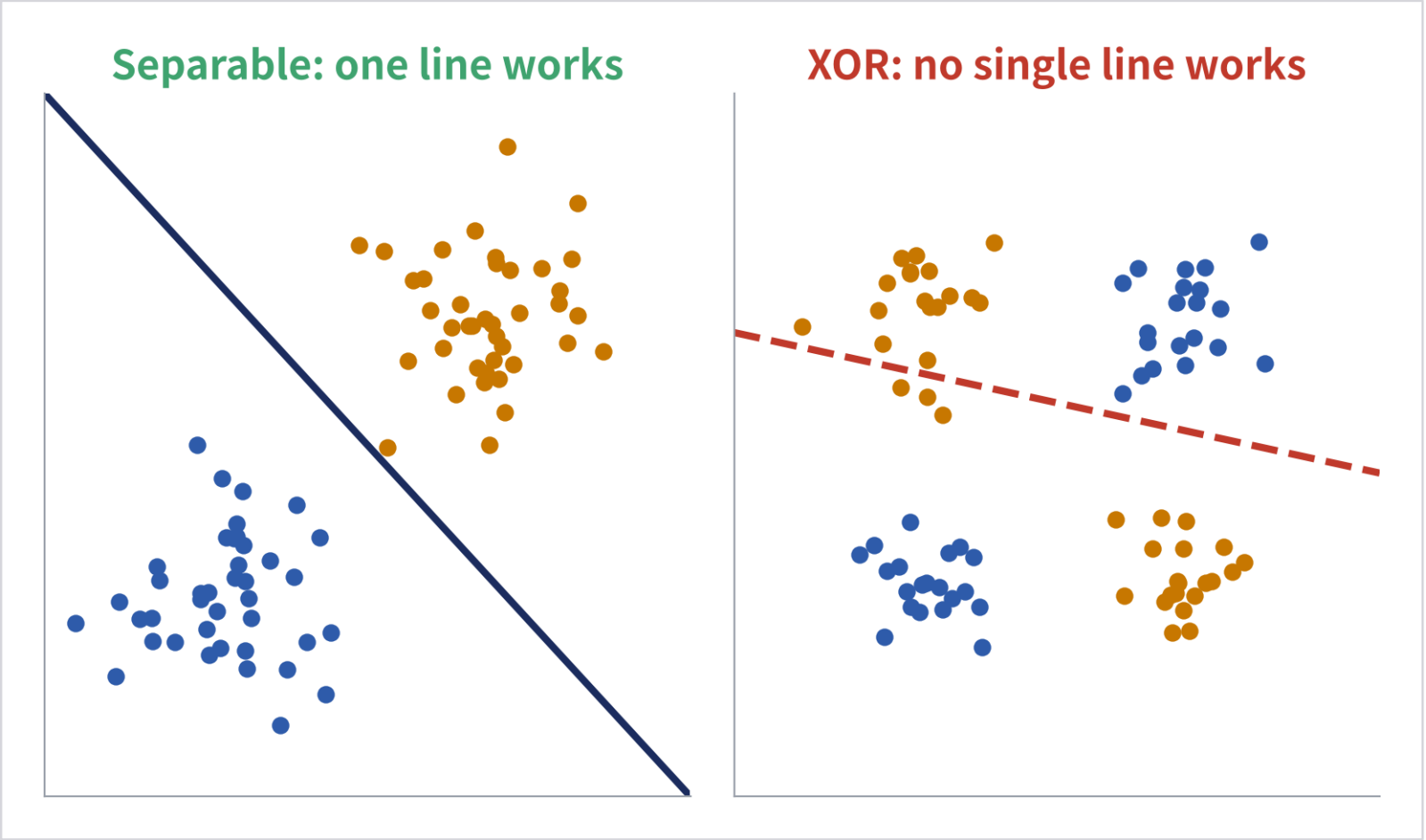
XOR = OR but not AND · everyday case: a hallway light with two switches

One neuron can only draw one straight boundary

XOR (exclusive OR): the class is positive when exactly one input is positive

What to see

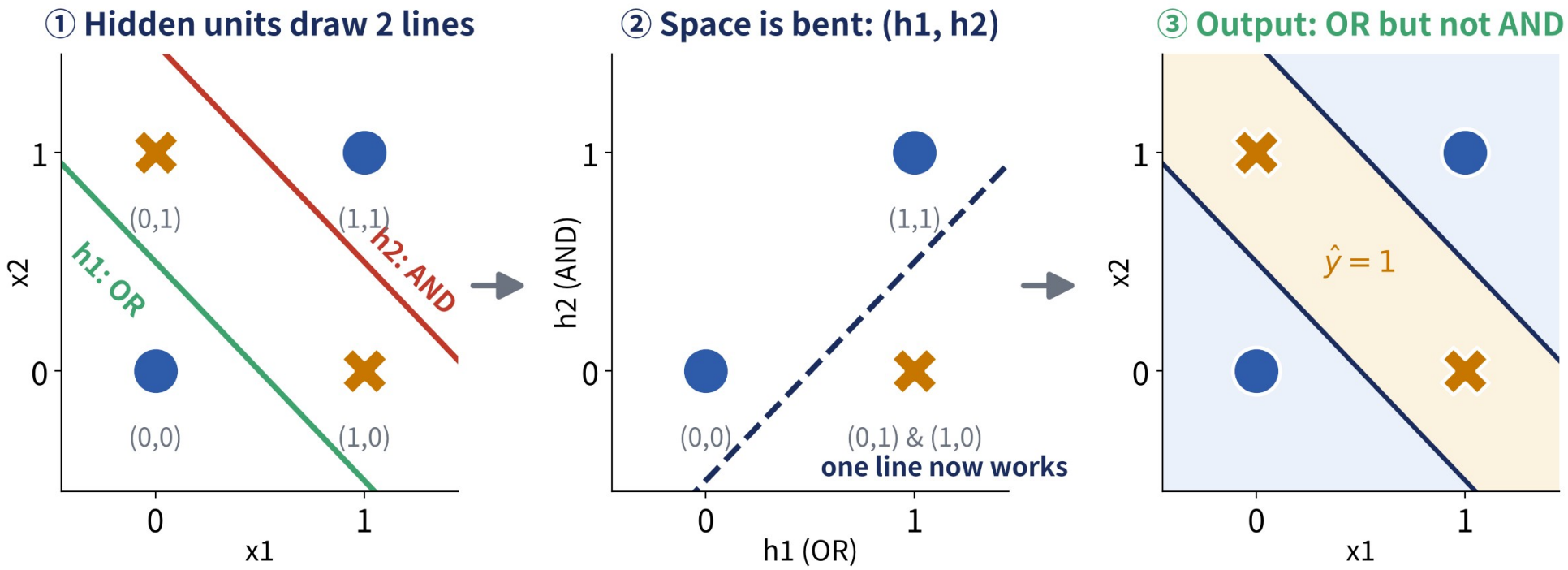
- Left: a line splits them
- Right: no line can
- Fix: add a hidden layer
- 1969 Minsky–Papert critique



Two hidden units bend the space so one line suffices

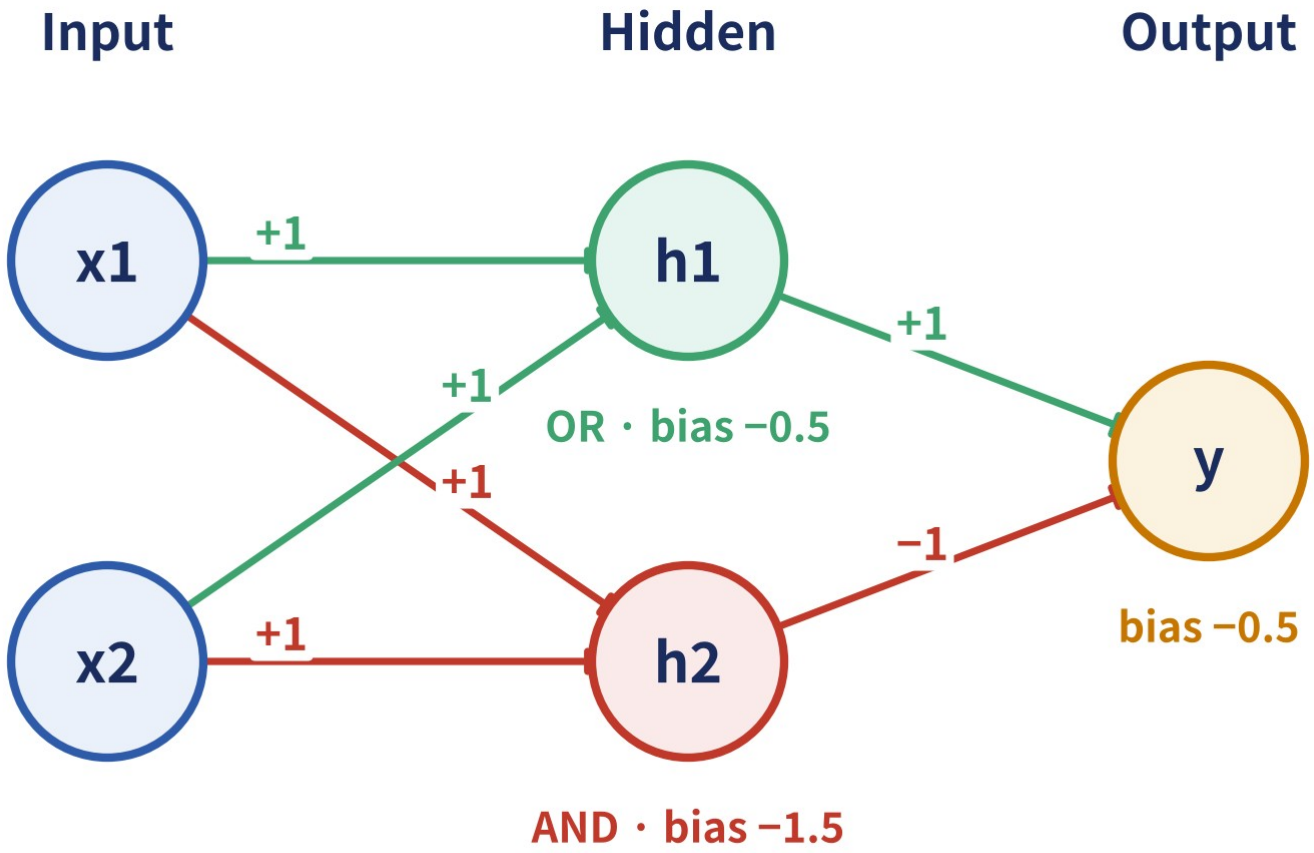
Hand-set weights; 1[·] = 1 if the condition holds, else 0 (step activation)

$$h_1 = \mathbf{1}[x_1 + x_2 - 0.5 > 0] \ \text{(OR)}, \qquad h_2 = \mathbf{1}[x_1 + x_2 - 1.5 > 0] \ \text{(AND)}, \qquad \hat{y} = \mathbf{1}[h_1 - h_2 - 0.5 > 0]$$



Two hidden units and one output solve XOR

Step activation; weights and biases are set by hand here, training would find them



every unit fires (= 1) when its weighted sum + bias > 0

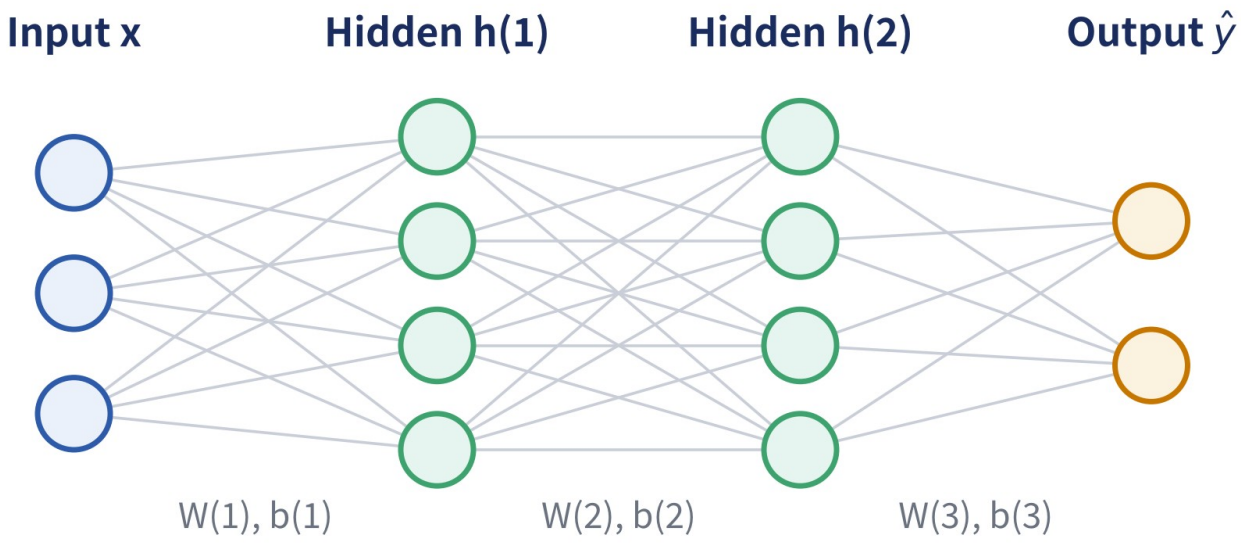
x1 ×2	h1 (OR)	h2 (AND)	ŷ
0 0	0	0	0
0 1	1	0	1
1 0	1	0	1
1 1	1	1	0

ŷ = OR − AND = XOR

Layers compose simple bends into complex functions

Multilayer perceptron (MLP): every unit in one layer feeds every unit in the next

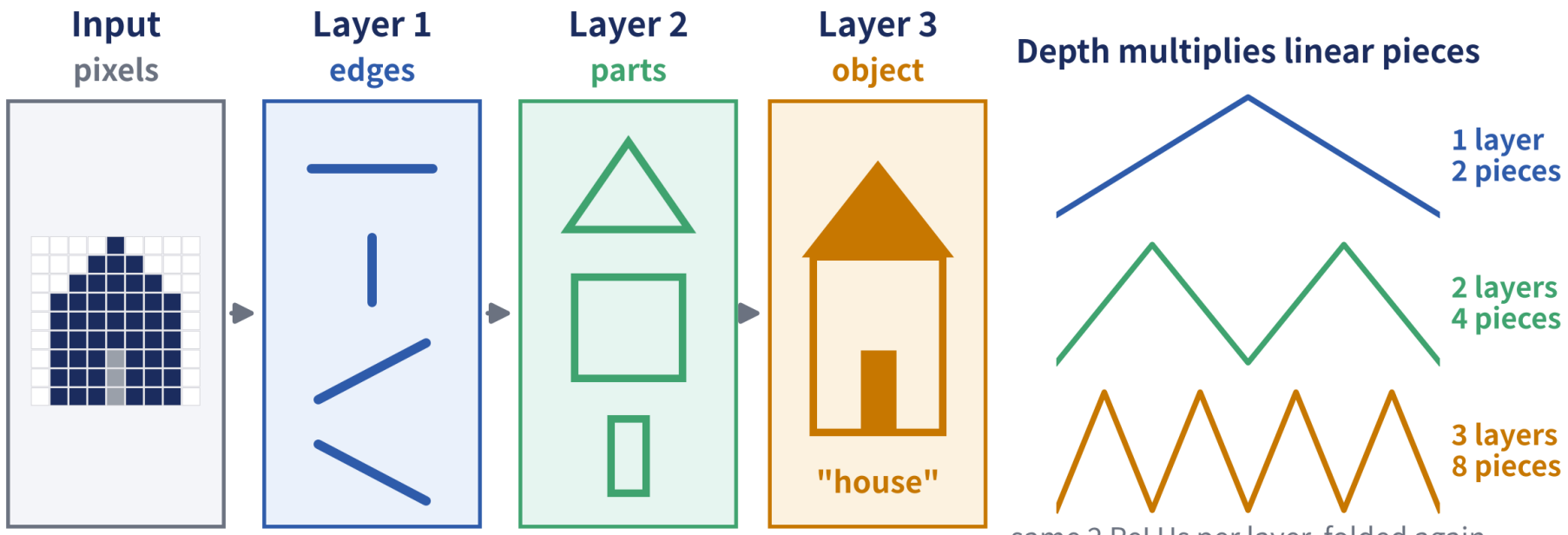
$$\mathbf{h}^{(1)} = \sigma\left(W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right), \quad \mathbf{h}^{(2)} = \sigma\left(W^{(2)}\mathbf{h}^{(1)} + \mathbf{b}^{(2)}\right), \quad \hat{y} = W^{(3)}\mathbf{h}^{(2)} + \mathbf{b}^{(3)}$$



Each layer builds richer features from the one below

A deep network is a composition of simple functions; depth stacks reuse, not just size

$$\hat{y} = f(\mathbf{x}) = \underbrace{f^{(3)}}_{\text{object}}\Big(\underbrace{f^{(2)}}_{\text{parts}}\Big(\underbrace{f^{(1)}}_{\text{edges}}(\mathbf{x})\Big)\Big)$$



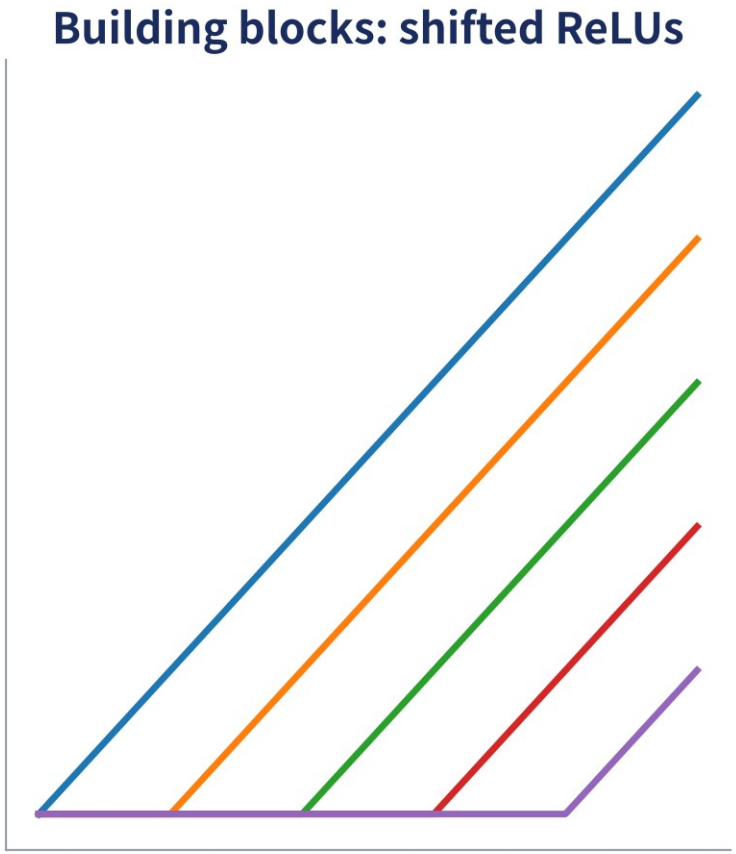
each layer combines the previous layer's features

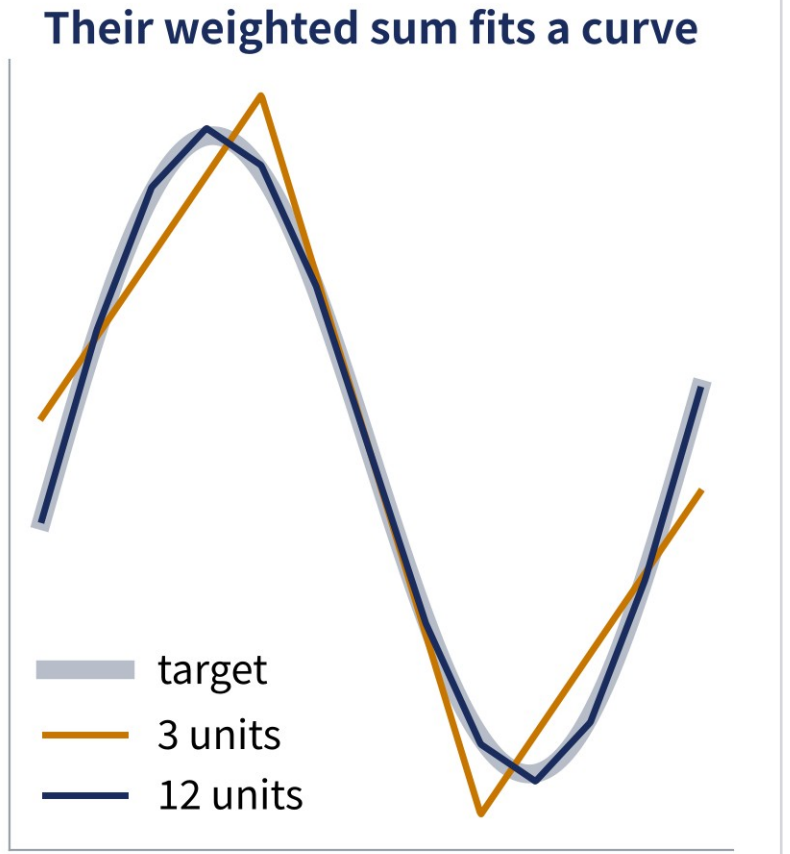
Enough hidden units can approximate any curve

Universal approximation theorem (Cybenko 1989; Hornik 1991)

Intuition

- Each ReLU adds a kink
- Weights set kink slopes
- More units, finer fit
- Says it exists, not how to find it





02

PART 2 · LEARNING

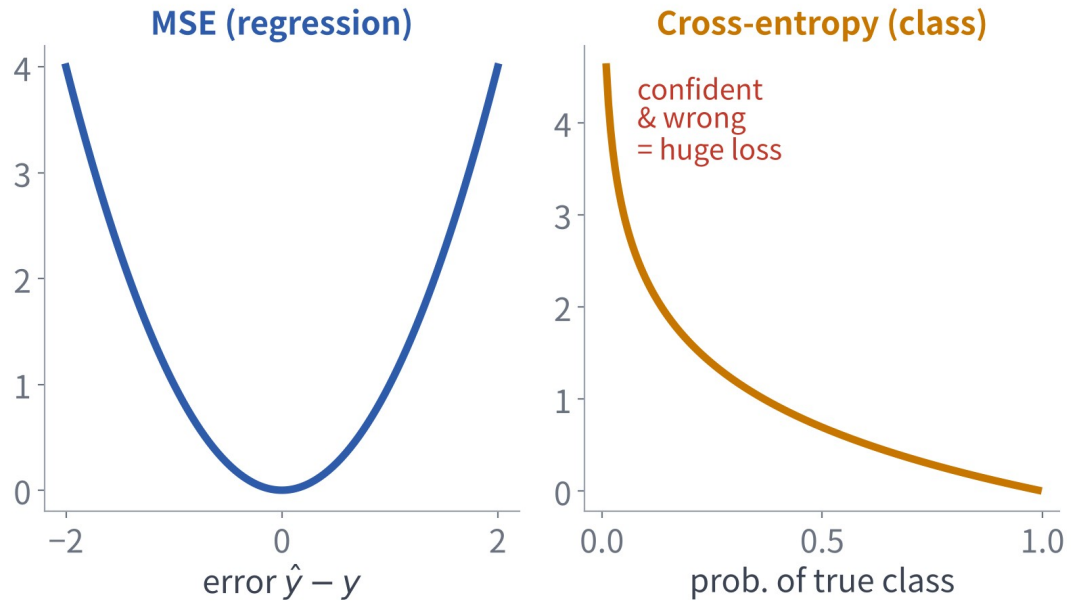
How a Network Learns

Loss, gradient descent, learning rate and backpropagation

A loss function scores how wrong the model is

Regression uses squared error; classification uses cross-entropy (log loss)

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2 \qquad \text{CE} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\right]$$



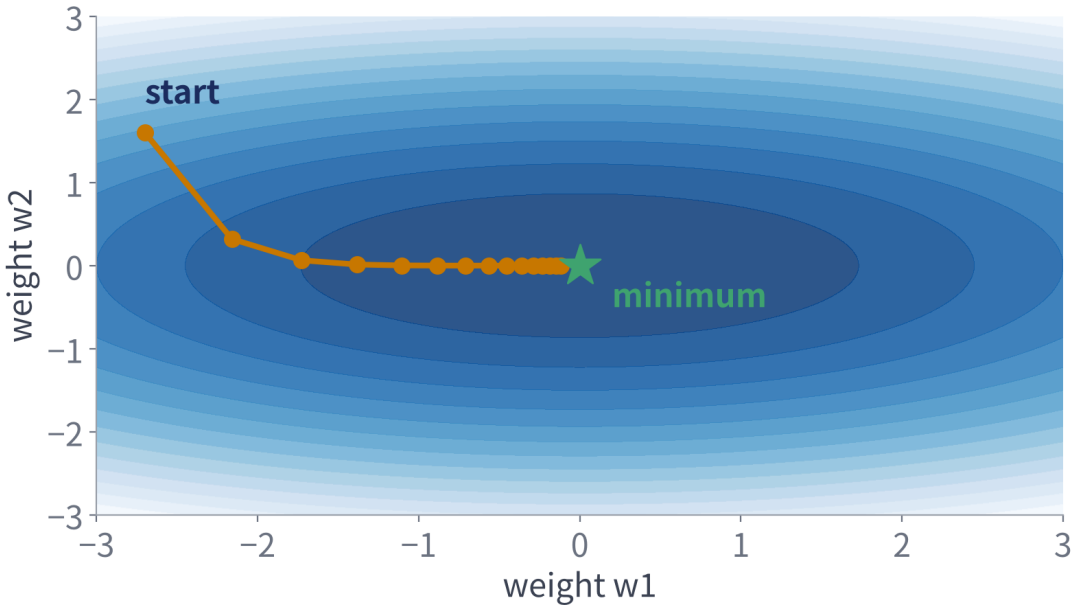
Why these two

- MSE punishes big misses hardest
- CE explodes when the model is confident and wrong
- Training = make this number small

Gradient descent repeatedly steps against the slope

θ (theta) = all weights and biases; η (eta) = learning rate

$$\theta \leftarrow \theta - \underbrace{\eta}_{\text{step size}} \underbrace{\nabla_\theta L(\theta)}_{\text{uphill direction}}$$



Hiker in fog

- Feel the slope under your feet
- Step downhill, then feel again
- Mini-batch SGD: slope from a small random sample

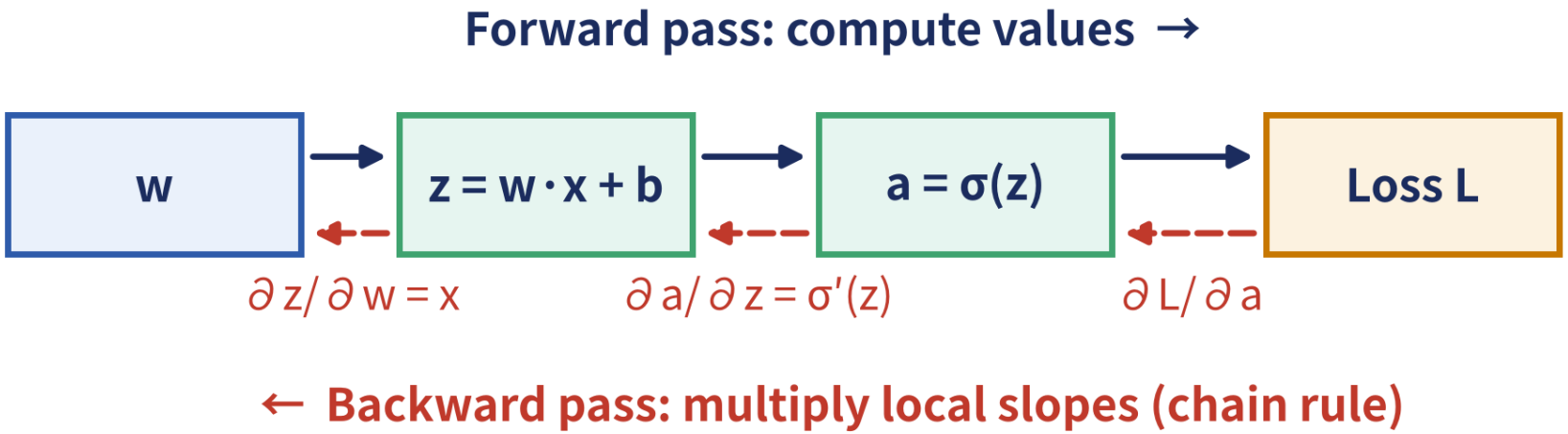
The learning rate decides whether training converges

Same loss $L(w) = w^2$, same start, three step sizes



Backprop sends blame backward via the chain rule

One forward pass computes values; one backward pass computes every gradient



$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Training repeats four steps until the loss stops falling

One pass over the whole dataset is an epoch; one mini-batch step is an iteration

1. Forward

predict

Push a mini-batch through the network to get ŷ

x → ŷ

2. Loss

measure

Compare ŷ with the true labels y

L(ŷ, y)

3. Backward

assign blame

Backprop computes ∂L/∂θ for every weight

chain rule

4. Update

step

Optimizer moves weights a small step downhill

θ ← θ − η∇L

Frameworks (PyTorch, JAX) automate steps 3–4; you design steps 1–2

03

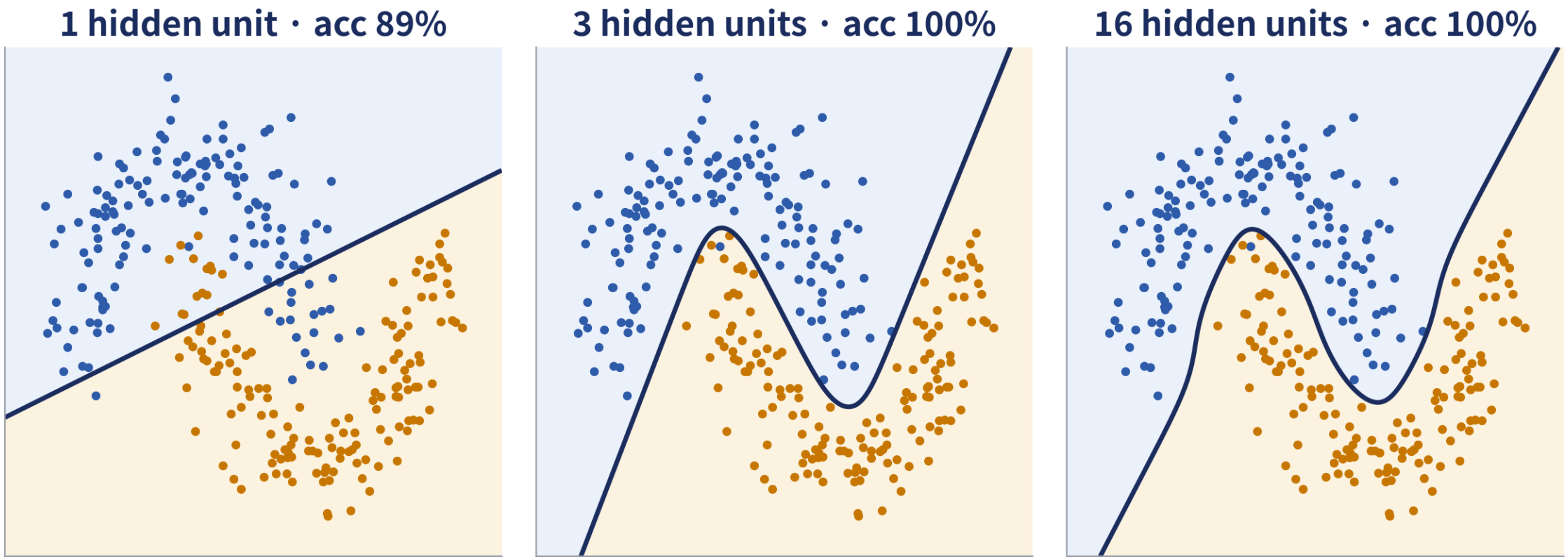
PART 3 · IN PRACTICE

Making Networks Work

Capacity, overfitting and sensible defaults

More hidden units let the decision boundary curve

Two-moons data, one hidden layer (tanh), trained by gradient descent in NumPy

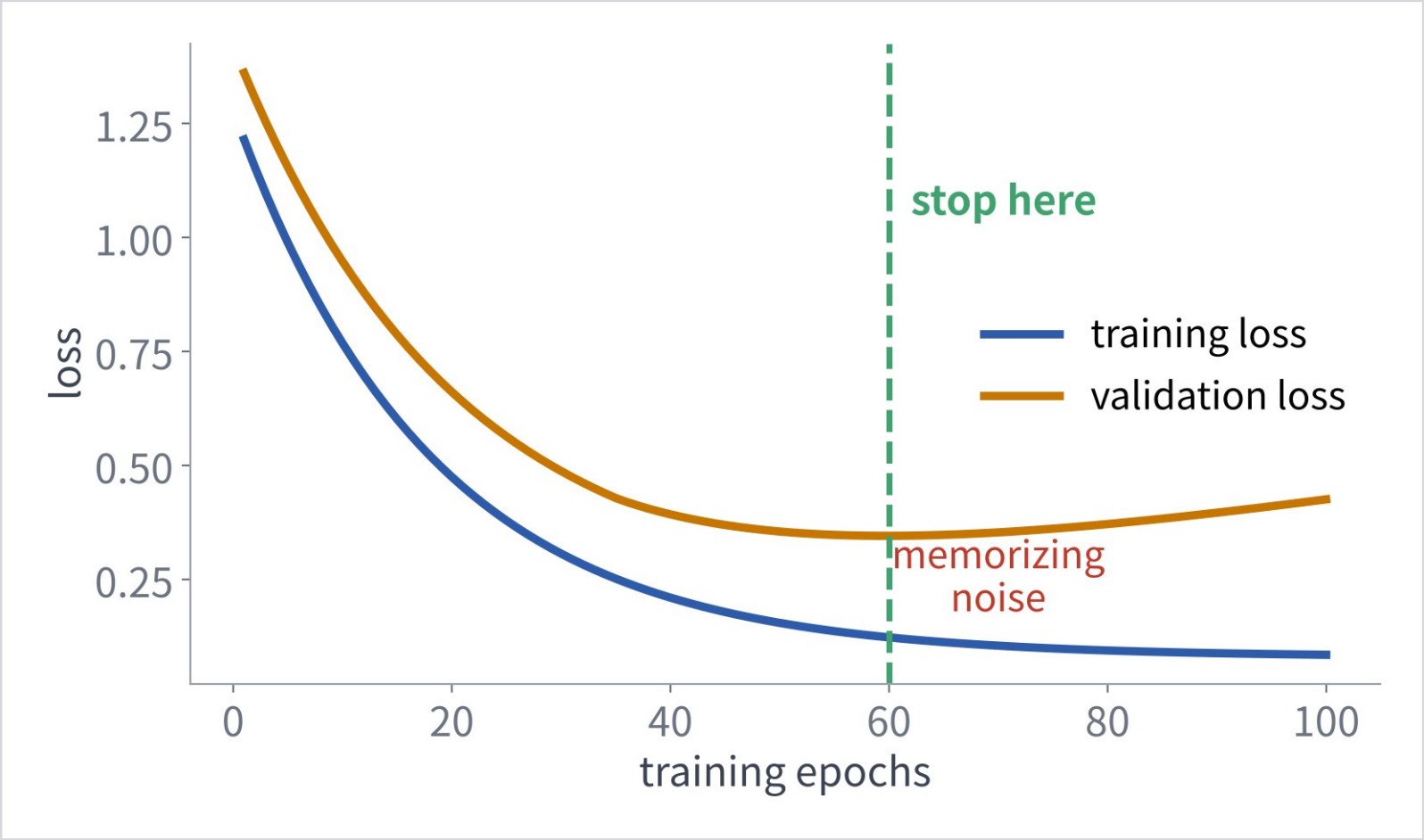


Validation loss reveals when memorizing begins

Training loss always falls; validation loss turns up once noise is being fitted

Remedies

- Early stopping
- More data or augmentation
- Weight decay (L2)
- Dropout



Sensible defaults get most networks training

Starting points, not rules — tune on the validation set

Choice	Default	Why
Hidden activation	ReLU (or GELU)	Cheap, no vanishing gradient for $z > 0$
Output + loss	Linear + MSE / softmax + CE	Matches the target type
Optimizer	Adam, $\eta \approx$ 1e-3	Adapts step size per weight
Initialization	He / Xavier	Keeps signal scale stable across layers
Regularization	Weight decay + early stopping	Controls overfitting cheaply

04

PART 4 · EXAMPLES

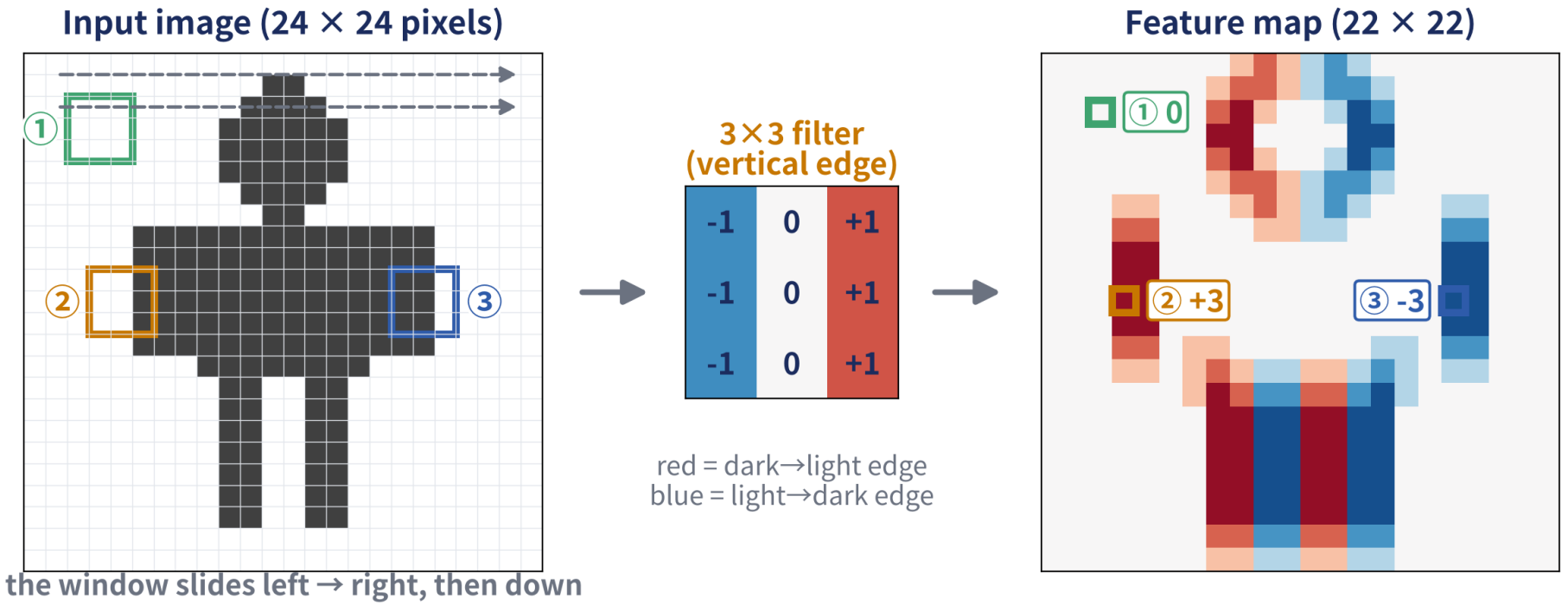
Modern Architectures and Finance

Same building blocks, different wiring: CNN, RNN, autoencoder, transformer

A sliding filter detects local patterns in images

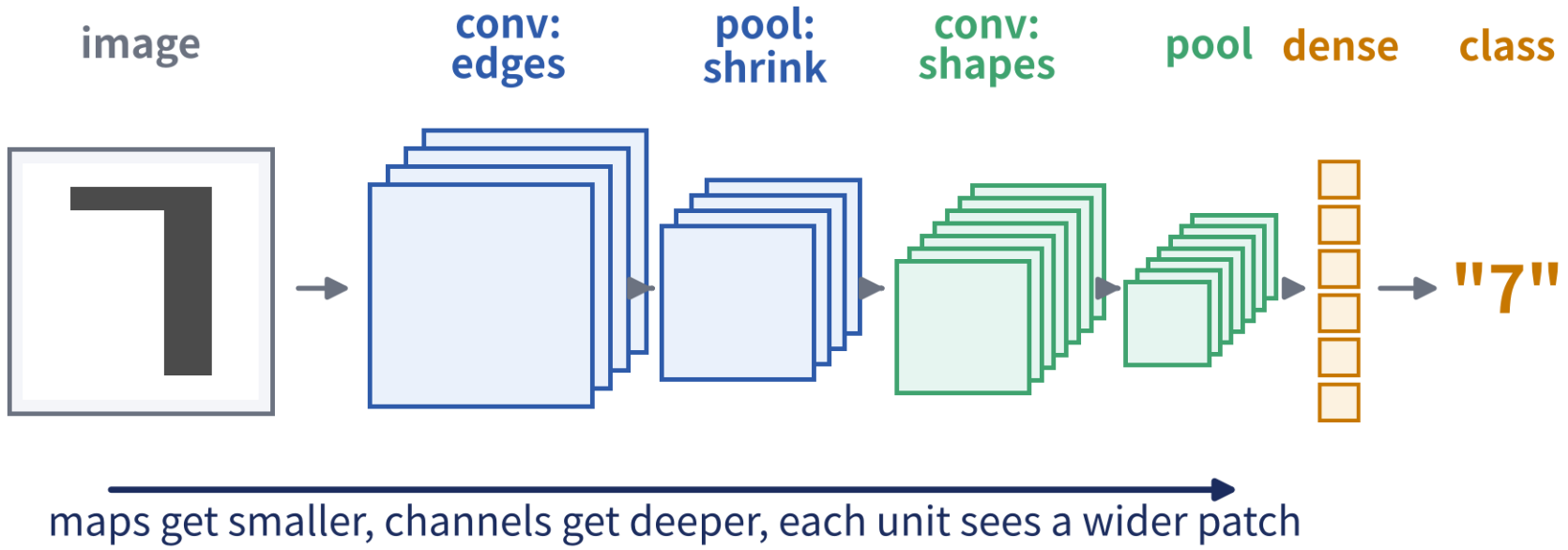
Convolution: the same 3×3 weights are reused at every position (weight sharing)

$$\text{feature}(i,j) = \sum_{m=0}^{2}\sum_{n=0}^{2} \text{image}(i+m,\ j+n)\cdot K(m,n)$$



Stacked convolutions turn pixels into concepts

Pooling halves each map; deeper layers see wider patches (receptive field)

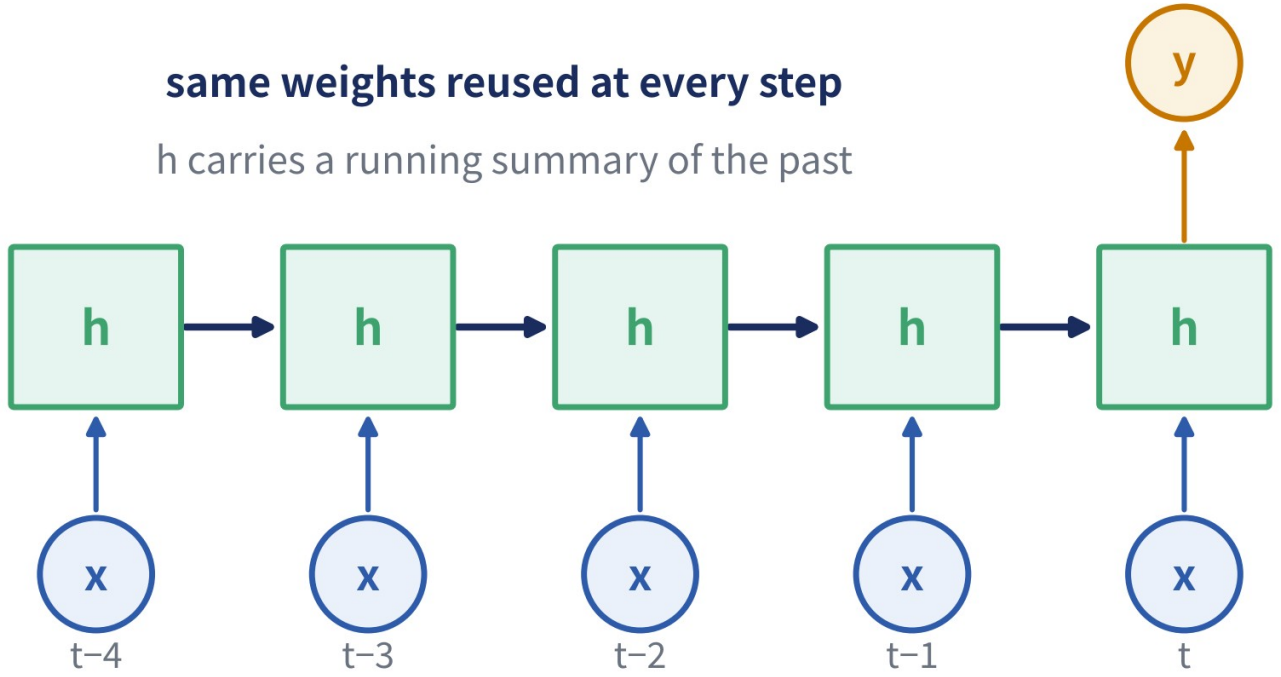


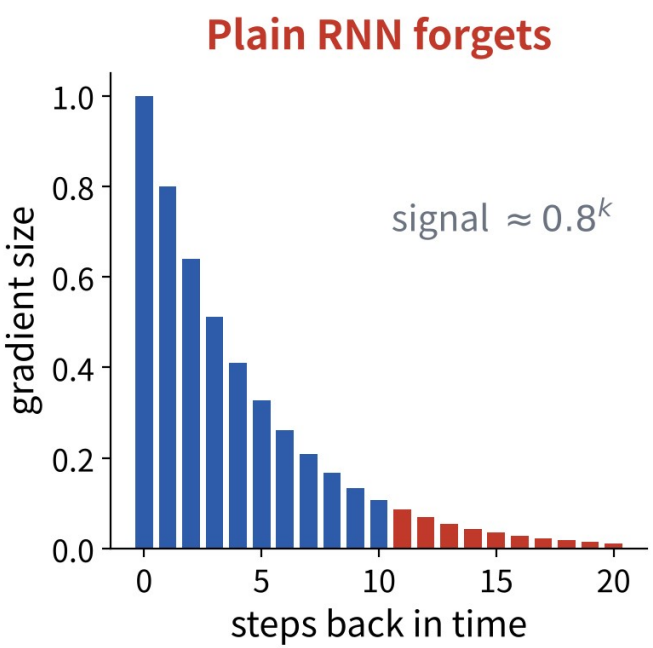
Finance: Jiang, Kelly & Xiu (2023) — a CNN on price-chart images predicts returns

A recurrent network carries memory from step to step

Time series and sequences: the order of observations matters

$$h_t = \tanh\left(W_h\, h_{t-1} + W_x\, x_t + b\right), \qquad \hat{y}_t = W_y\, h_t$$

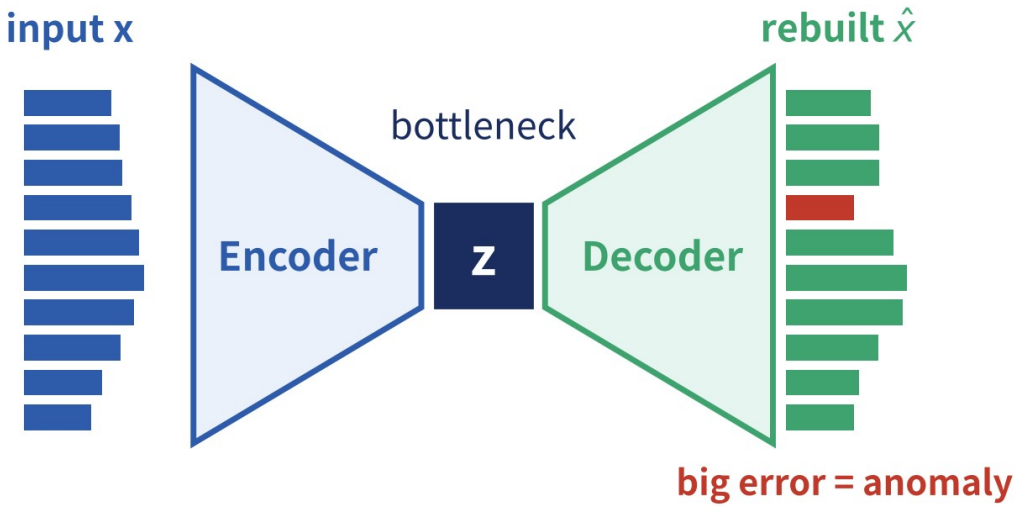




Autoencoders compress data through a bottleneck

The target is the input itself, so no labels are needed (unsupervised learning)

$$\mathbf{z} = f_{\text{enc}}(\mathbf{x}), \qquad \hat{\mathbf{x}} = f_{\text{dec}}(\mathbf{z}), \qquad \min \|\mathbf{x} - \hat{\mathbf{x}}\|^2 \quad (\dim \mathbf{z} \ll \dim \mathbf{x})$$



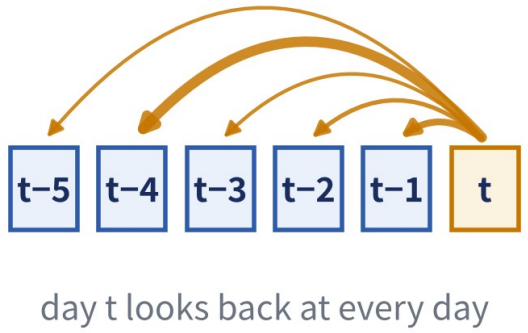
Finance use

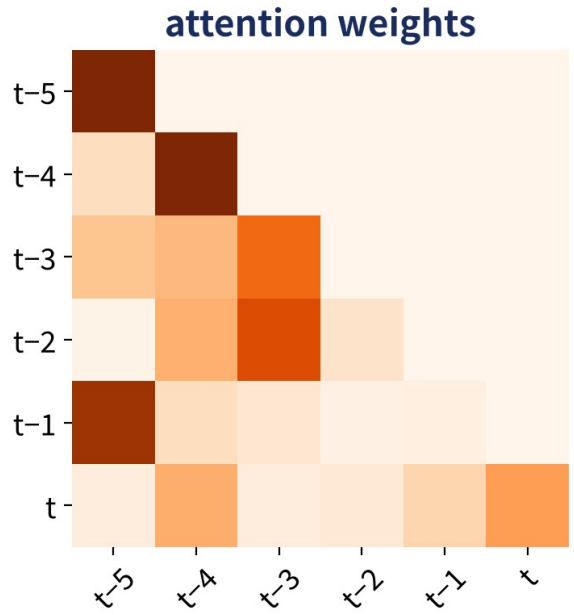
- Yield curve: z ≈ level, slope, curvature (nonlinear PCA)
- Fraud and AML: high reconstruction error flags odd trades

Self-attention lets each step weigh all others

Self-attention: each token scores relevance to all others, then takes a weighted average

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \qquad Q = XW_Q,\ K = XW_K,\ V = XW_V$$



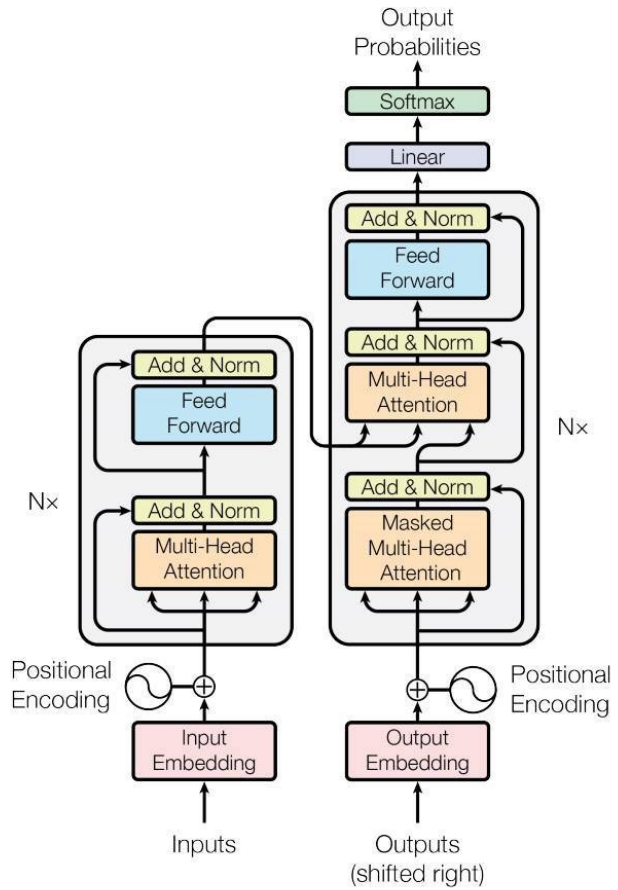


Finance use

- Text: FinBERT-style sentiment on filings and news
- Sequences: attend to the relevant past days, not only the last few

The original transformer pairs encoder and decoder

Figure 1 of Vaswani et al. (2017), Attention Is All You Need



How to read the figure

- Left tower = encoder: self-attention over the whole input
- Right tower = decoder: masked attention sees only earlier outputs
- Its middle attention block queries the encoder (cross-attention)
- Add & Norm = residual shortcut + layer normalization
- Positional encoding injects word order; N = 6 layers
- BERT keeps the encoder; GPT keeps the decoder

Each finance task maps to a matching network

Published examples; architecture names refer to Parts 1–4 of this lecture

Problem	Architecture	What the network does
Return prediction	MLP	Gu, Kelly & Xiu (2020): beats linear models on firm characteristics
Factor model	Autoencoder	Gu, Kelly & Xiu (2021): latent factors, betas from characteristics
Fraud / AML	Autoencoder	Flags transactions it cannot reconstruct well
Text signals	Transformer	FinBERT (Araci 2019): sentiment from financial text
Hedging	MLP policy	Deep hedging (Buehler et al. 2019): learns hedges under costs

A network is a flexible function fitted downhill

1

Neuron = weighted sum + activation

The nonlinearity is what makes depth worth having.

2

Layers = composed bends

Width and depth set how complex a boundary the model can draw.

3

Training = loss + backprop + gradient descent

Watch validation loss to know when to stop.

Every architecture = neurons + an objective + gradient descent